Pulseq Principles and Basics

Maxim Zaitsev

Division of Medical Physics, Dept. of Radiology,
University Medical Center Freiburg, Germany

February 2026

Downsides of proprietary MR method development

- Global disadvantages for industry
  - Repeated effort for replication of the MR methods and reconstruction algorithms
  - IP clashes and disproportional secrecy
- Global disadvantages for research
  - Personnel transition between platforms is challenging (months of re-learning)
  - Cooperation between sites is hindered
- Replication crisis due to secrecy, incomplete documentation, software updates…
  - Global disadvantages for healthcare
  - Slow dissemination of novel ideas and cutting-edge technology

Traditional approach to sequence development

- Attend vendor's scanner programming course
  - Waiting time + costs
- Acquire product source code for a starting point
  - Several weeks or … years…
- C++ sequence sources from another research site?
  - Sequences based on vendor's product protocols?..
- OK for clinical studies or Ph.D. theses
  - Incompatible with master and bachelor projects

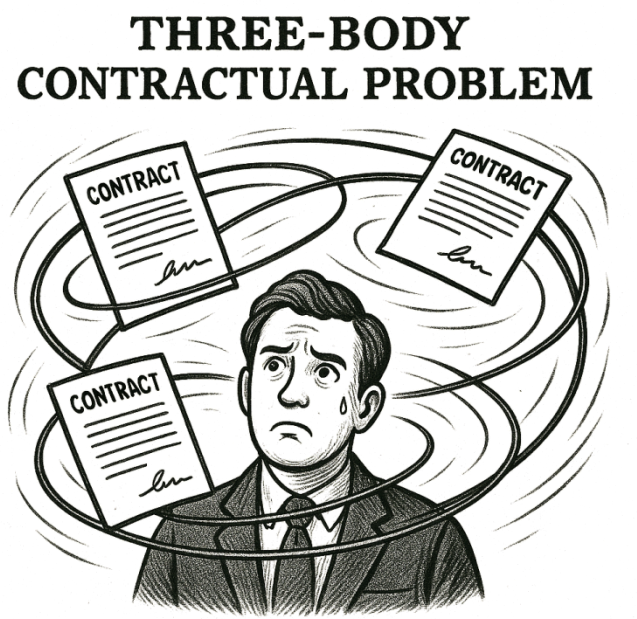


illustration by chatGPT

Pulseq Philosophy

- Minimize development and support effort for us…
  - Lean sequence-to-hardware interface
- Remove barriers in sequence programming
  - Make simple things truly simple
  - Comfortable learning curve for advanced topics
- Make researcher-oriented features accessible
  - Arbitrary gradients, arbitrary RF, free ordering, X-nuclei, …
  - Tools for automatic k-space calculation, sequence validation,…
- Serve the entire MR field
  - From basic MR education to cutting edge experiments
- **Promote sharing, exchange and reproducibility!**

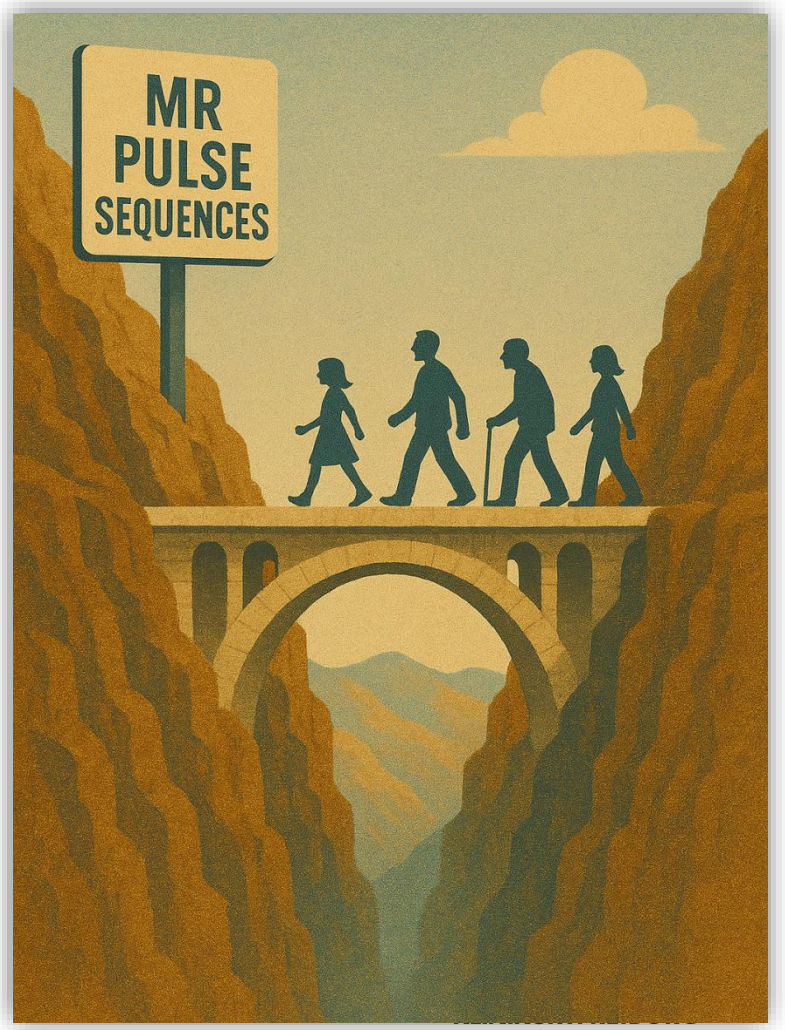


illustration by chatGPT

What is *Pulseq*?

- *Pulseq* is a language to describe MR pulse sequences

- *Pulseq* is the software to generate such pulse sequence descriptions

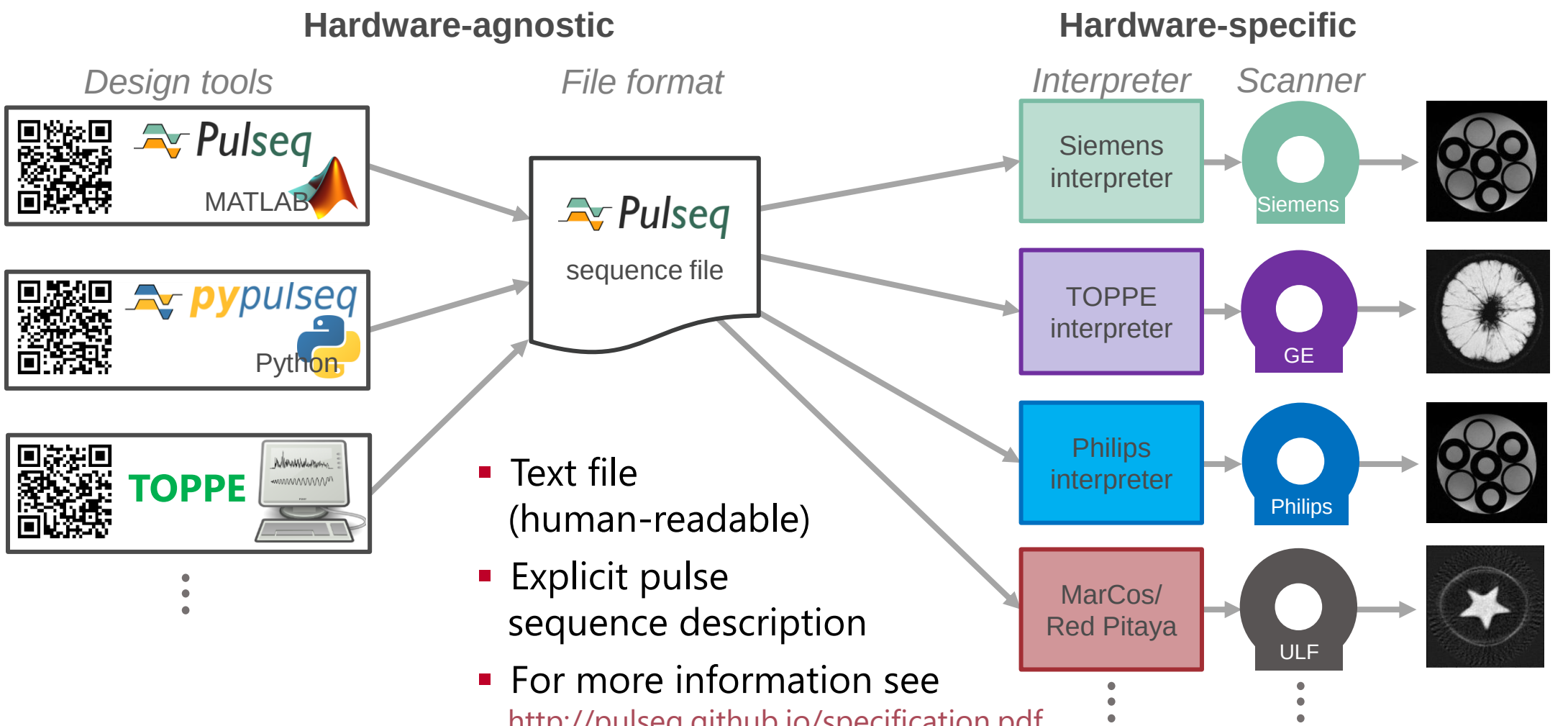
- *Pulseq* sequences are fixed successions of RF and gradient pulses, and ADC events

- *Pulseq* scripts can re-generate Pulseq sequences to accommodate user input

Pulseq ecosystem includes sequences, software to generate them and software and hardware to consume them

graphics by chatGPT

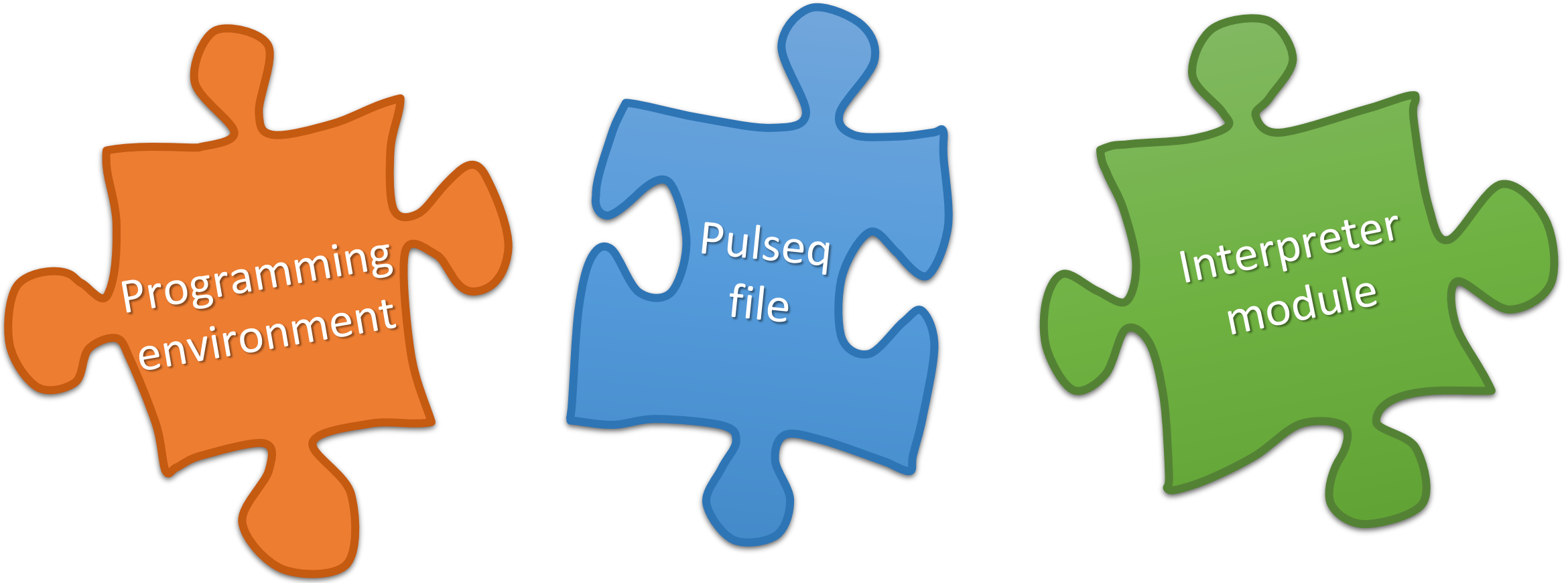
Pulseq framework overview



- Text file (human-readable)
- Explicit pulse sequence description
- For more information see http://pulseq.github.io/specification.pdf

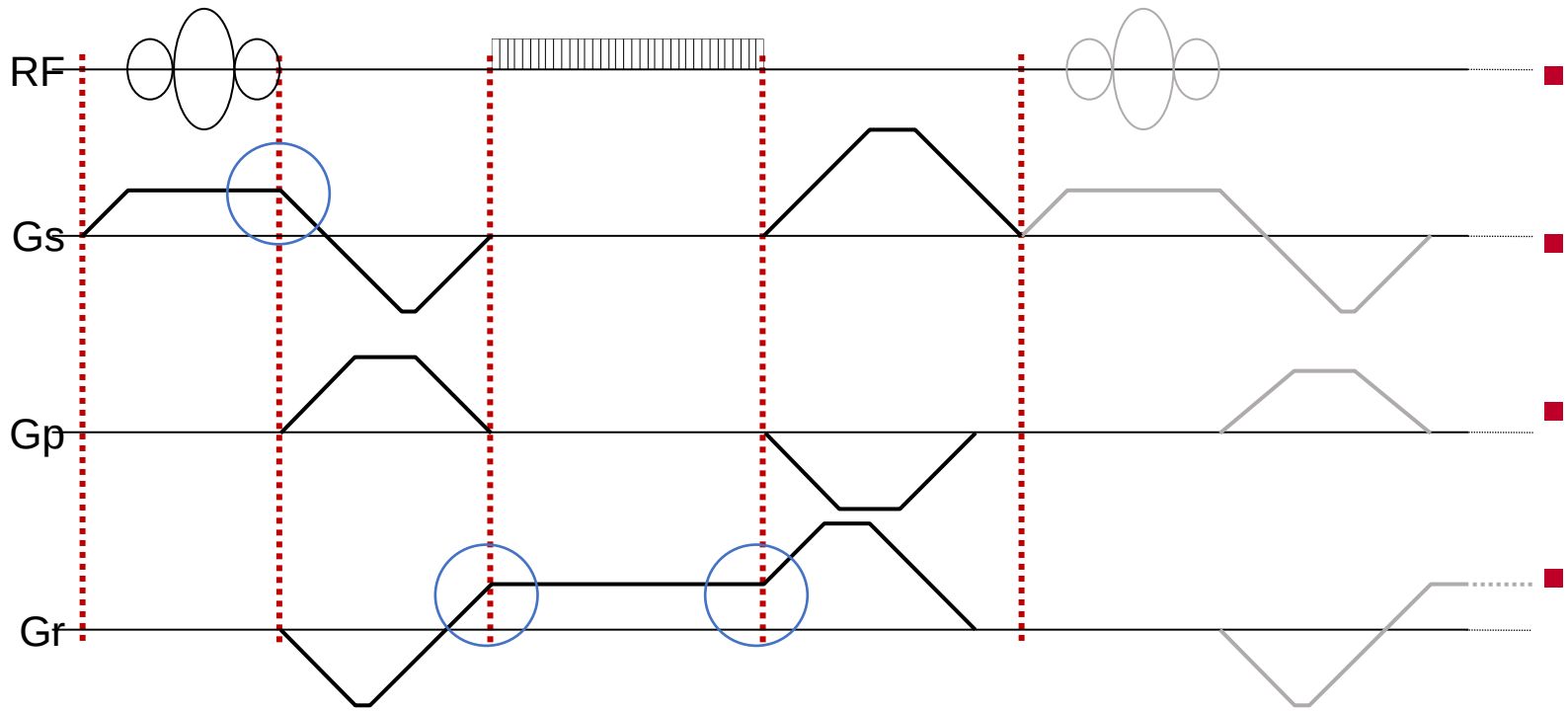
UNIVERSITÄTS KLINIKUM FREIBURG

Pulseq: pieces of the puzzle



Pulse sequence definition in *Pulseq*

Concatenation of non-overlapping blocks



- Block 1:
  gradient and RF
- Block 2:
  only gradients
- Block 3:
  gradient and ADC
- Block 4:
  only gradients
- Block 5:
  gradient and RF …

Gradients do not have to start or end at 0 at the block boundaries

see http://pulseq.github.io/specification.pdf for more details

Pulseq block concept in detail

- Each block may contain following events:
  - One optional gradient pulse per axis
  - One optional RF pulse
  - One optional ADC event
- Individual events may define own start delays
- All events in the block overlap in time
- Duration of the block is defined by the longest event
  - **Required block duration** can be specified as a number

- Explicit sequence description
  - No loops, no dependent parameters*

* Recent extensions (ONCE label, soft delays,…) introduce some variability

How to design a sequence in *Pulseq*

conceptual design steps

- Step 1: spilt the time axis into blocks
  - RF and ADC objects **cannot cross** block boundaries
  - Gradients **may remain non-zero** at the edges of the block
- Step 2: assign events to the blocks

practical implementation steps

- Step 3: create/calculate all events
- Step 4: populate the blocks and add them to the sequence

validation steps

- Step 5: check timing, verify k-space trajectory, hardware and PNS limits, mechanical resonances, etc

Matlab *Pulseq* workflow

```
fov = 256e-3; slThck = 5e-3;
Nx = 128; Ny = Nx;
TE = 10e-3; TR = 20e-3; alpha=15;
```

```
sys = mr.opts('MaxGrad',20,'GradUnit','mT/m',...
    'MaxSlew',100,'SlewUnit','T/m/s',...
    'rfRingdownTime', 20e-6, 'rfDeadtime', 100e-6, 'setAsDefault', true);
seq=mr.Sequence(sys);
```

```
[rf, gz, gzReph] = mr.makeSincPulse(alpha*pi/180, 'Duration', 4e-3,...
    'SliceThickness', slThck, 'apodization', 0.5,'timeBwProduct', 4);

gx = mr.makeTrapezoid('x', 'FlatArea', Nx/fov, 'FlatTime', 6.4e-3);
adc = mr.makeAdc(Nx, 'Duration', gx.flatTime, 'Delay', gx.riseTime);
gxPre = mr.makeTrapezoid('x', 'Area', -gx.area/2, 'Duration', 2e-3);
gyPre = mr.makeTrapezoid('y', 'Area', Ny/2/fov, 'Duration', 2e-3);
```

```
phaseAreas = (0:Ny-1)/(Ny/2)-1;
delayTE = round((TE - mr.calcDuration(gxPre) - mr.calcDuration(gz)/2 ...
    - mr.calcDuration(gx)/2)/seq.gradRasterTime)*seq.gradRasterTime;
delayTR = round((TR - mr.calcDuration(gxPre) - mr.calcDuration(gz) ...
    - mr.calcDuration(gx) - delayTE)/seq.gradRasterTime)*seq.gradRasterTime;
```

```
for i=1:Ny
    seq.addBlock(rf, gz);
    seq.addBlock(gxPre, mr.scaleGrad(gyPre,phaseAreas(i)), gzReph);
    seq.addBlock(delayTE);
    seq.addBlock(gx, adc);
    seq.addBlock(delayTR);
end
```

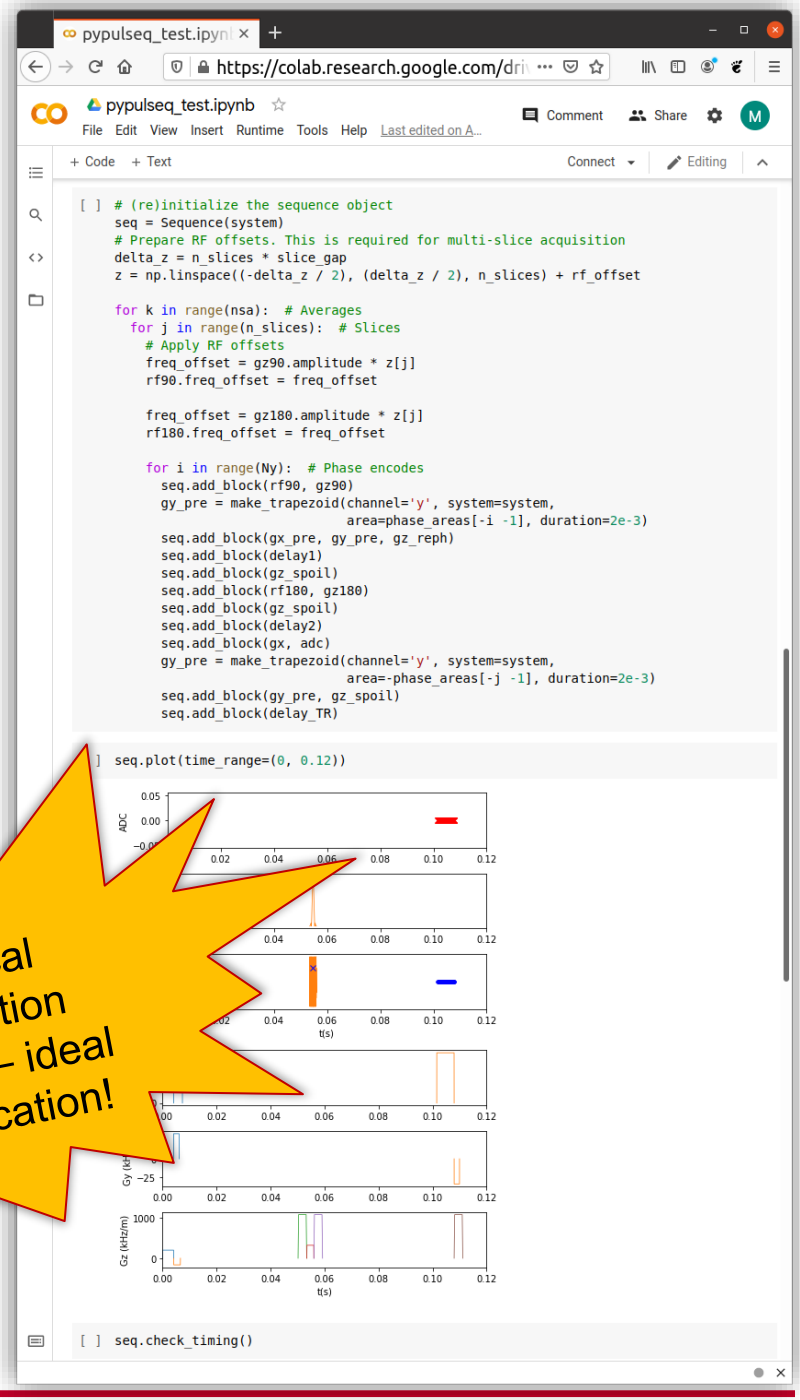
```
seq.setDefinition('FOV', [fov fov slThck]);
seq.setDefinition('Name', 'mini_gre');

seq.write('mini_gre.seq')
```

a runnable gradient echo sequence code (similar to Siemens' example miniFlash)

- Define high-level parameters (convenience)
- Define the system properties
- Define pulses and ADC objects used in the sequence
- Calculate the delays and reordering tables
- Loop and define sequence blocks
  - Duration of each block is defined by the duration of the longest event
- Prepare sequence export and write it out
- *Copy '*.seq' to the scanner and run it!*

PyPulseq workflow

- *PyPulseq* is a close replica of the original Pulseq toolbox that does not require a MATLAB license
- Runs in many Python environments, e.g. as notebook in Jupyter (http://jupyter.org/) or Google Colaboratory
- Identical workflow:
  - Define the system properties
  - Define high-level parameters (convenience)
  - Define pulses and ADC objects used in the sequence
  - Calculate the delays and reordering tables
  - Loop and define sequence blocks
  - *Download '*.seq' to the scanner and run it!*



No local installation needed – ideal for education!

Pulseq Tutorials

https://github.com/pulseq/tutorials

- Sequences, raw data and reconstruction scripts (Matlab)
- Some basic explanations

https://github.com/pulseq/ISMRM-Virtual-Meeting--November-15-17-2023

- Slides of all talks, much of teaching material, videos are available to the ISMRM members or registered participants
- Sequences (Matlab), recon, some data (dropbox)

https://github.com/pulseq/MR-Physics-with-Pulseq

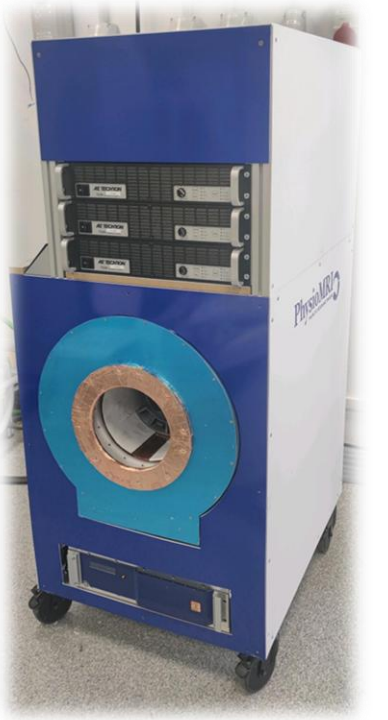
- Slides of all talks
- Demo code in PyPulseq, no installation needed (Google Colab)
- Sequences + recon, exercises, simulated with MR0, some raw data

Pulseq on Siemens platforms

- Over 200 C2P sites
- *Pulseq* works well on:
  - All Numaris4 platforms (tested on vb15…ve12u) and numerous hardware platforms (Symphony, Trio, 7T, 3T Connectom, Skyra, Prisma,Cima.X…)
  - All released NumarisX versions (xa11, xa20, xa30, xa40, xa5x, xa6x …)

Pulseq on Other Platforms

- Works very well on GE (thanks to Jon-Fredrik Nielsen)
- Philips interpreter (thanks to Tom Roos, Utrecht)
- Works on United Imaging
- Reported to work on Canon
- Nexus console (portable MRI) → → → → → →
- MaRCoS / Red Pitaya / Portable System → → → → → → → →



Pulseq Workflow

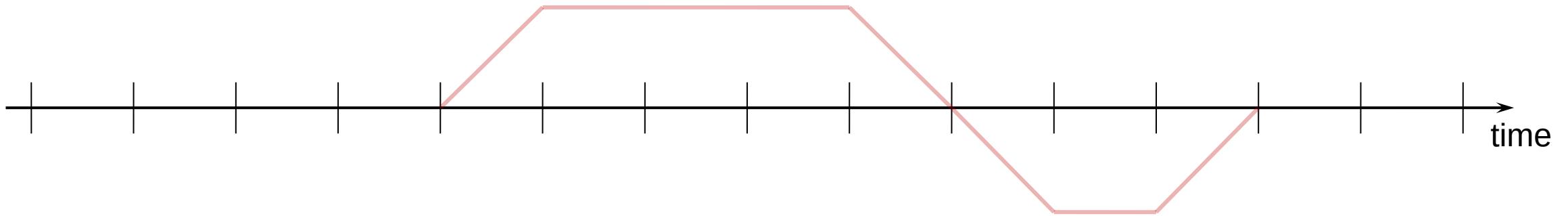
- Pulseq sequence (.seq file) is a **recording of your sequence**
  - Like a .mp3 or .slac music file
- It is **impossible** to change any aspects of the sequence related to its **timing or structure**
  - TE, TR, TI, matrix size, etc **are fixed**

  ➔ Pre-install various versions of your sequence with different parameters
- On-scan prescription (depending in the interpreter)
  - **FOV translation and rotation**
  - FOV scaling (by scaling all gradients on the corresponding axis)
  - Scale all or some flip angles / RF amplitudes (Siemens)
  - Change ‘soft delays’ (e.g. for cardiac triggering optimization)

Technical Details

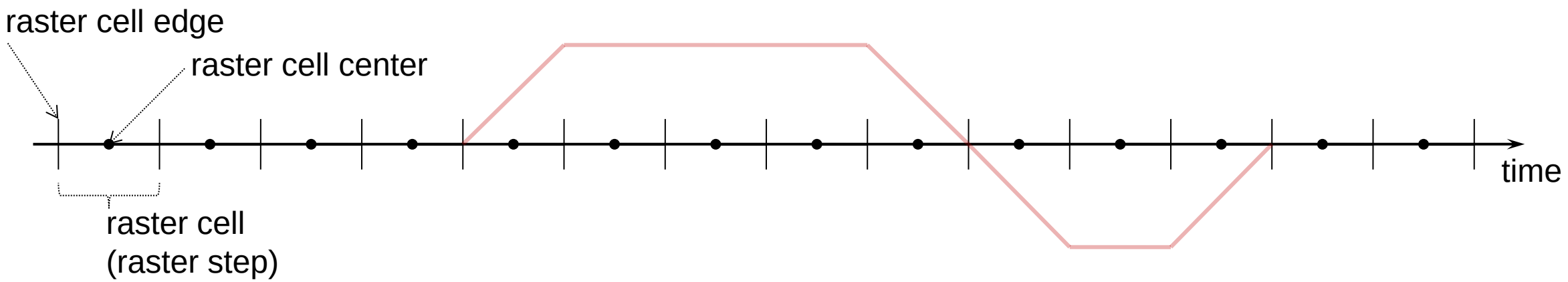
Timing and Shape Definitions

Shapes and Raster Times in *Pulseq*



- Specification needed for achieving precise control of gradient and RF waveforms (was missing prior to 1.4.0)
- Pulseq defines four types of raster times
  - adcRasterTime, rfRasterTime, gradRasterTime, blockDurationRaster
- Raster ‘thinking’ is probably one of the most demanding concepts in the practical pulse sequence programming
- Important concepts: **raster cells, edges and centers**

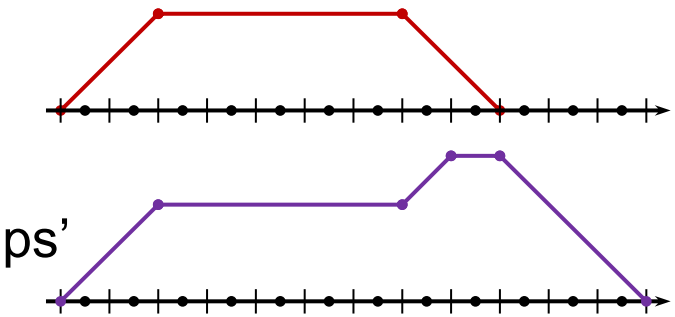
Timing and Shape Definitions



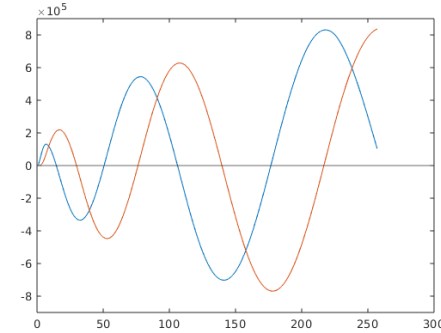
- The continuous and uninterrupted time axes is split in discrete time intervals, ***raster cells***, each of the equal duration of ***raster step***
- Each raster cell begins with the ***raster edge***
- The center of each raster cell is termed ***raster center***
- Consistent timing definitions for gradients, RF and ADCs
  - See https://pulseq.github.io/pulseq_shapes_and_times.pdf for details

ArbitraryGrads vs. ExtendedTraps

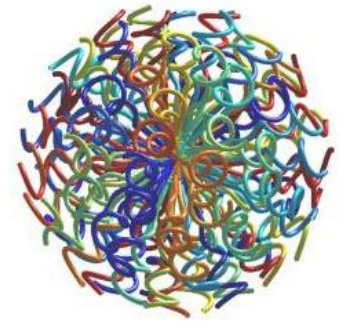
- Conventional trapezoid gradients
- Extended Trapezoids
  - On Siemens: constructed on the fly from overlapping triangle ‘blips’



- Arbitrary gradients
  - Require more memory at run-time
  - On Siemens: all shapes need to be prepared before the start of the scan to avoid hardware instabilities ➔ slower loading and starting time



- Optimization strategy in Pulseq sequence design
  - Reduce the number of <u>different arbitrary gradients</u> (e.g. by selecting appropriate block boundaries)
  - <u>Rotation Extension</u> to re-use gradient shapes

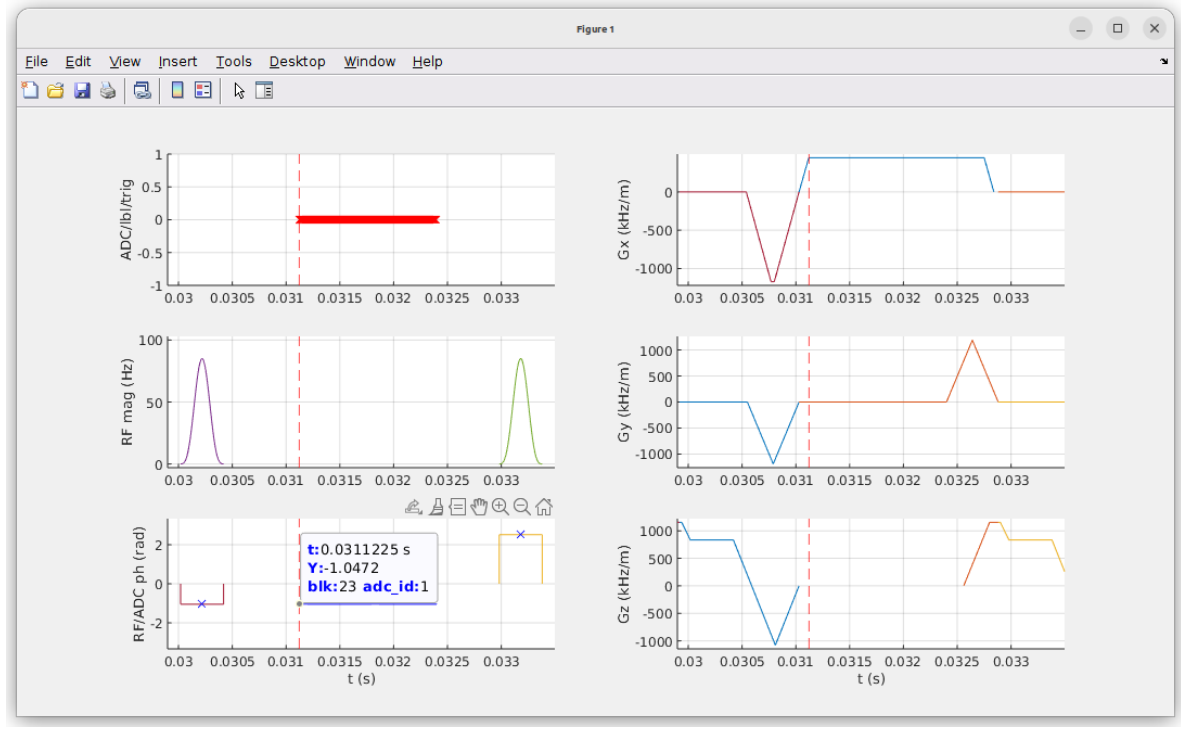


helical cones trajectory

Visualisation & Debugging Tools

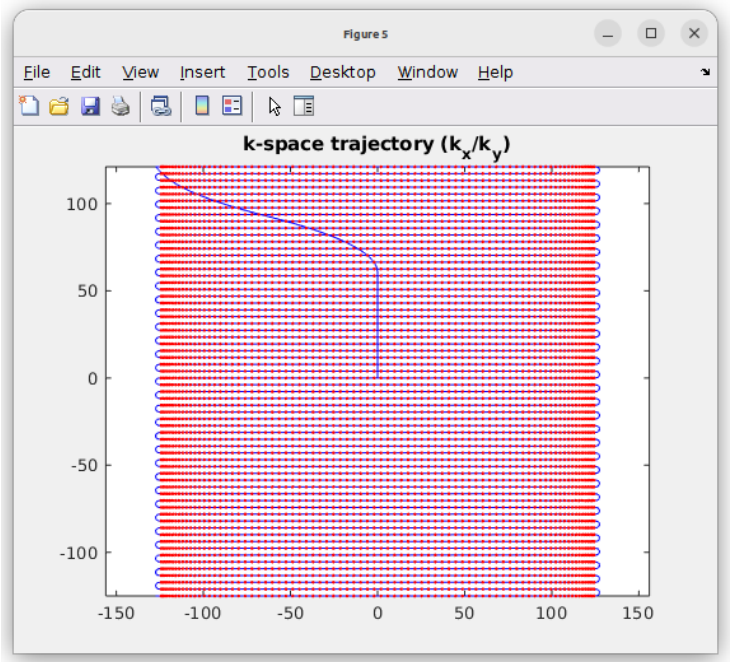
Detailed Display Options

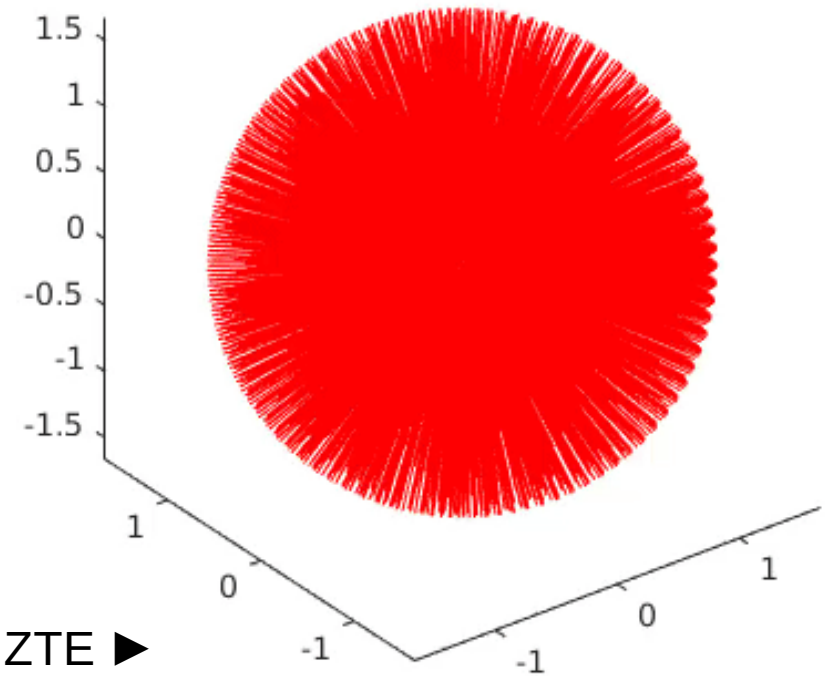
- Plot and examine sequence timing
  - Each Pulseq object is plotted in different color
    - Visualization of extended trapezoids split across blocks
  - Phase of the RF pulse center
  - Time and phase of each ADC sample
  - Detailed tool-tips with block and object information



K-Space Calculation in 2D and 3D

- Use seq.calculateKspacePP()
  - Consistent k-space trajectory information
  - Powerful debugging tool
  - Links to image recon toolboxes





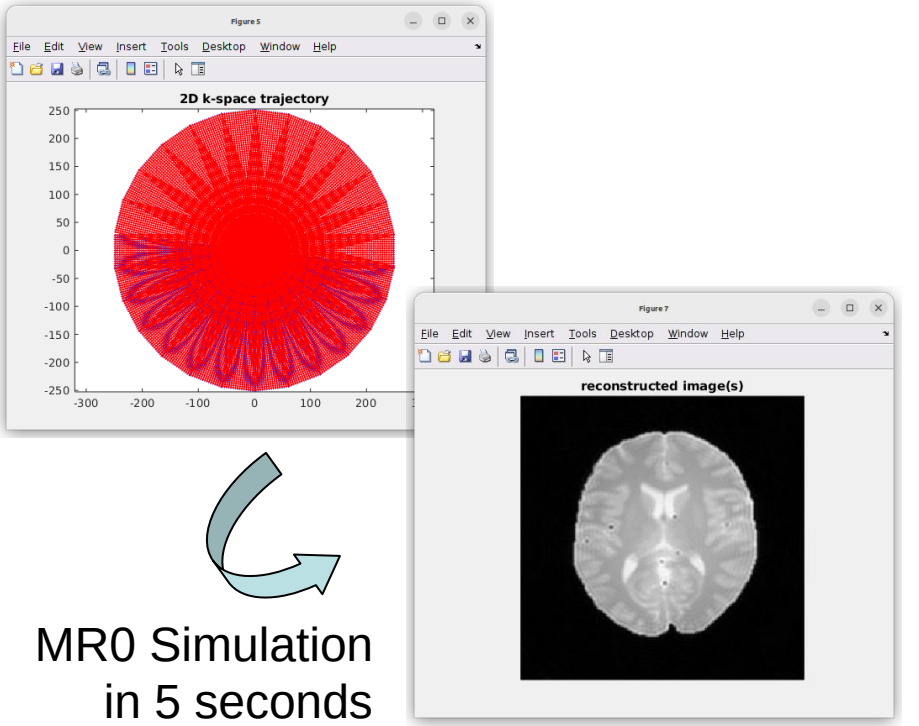
3D k-Space of ZTE ►

◄ 2D k-Space of EPI

Pulseq & Signal Simulation

- Numerous MR signal simulators accept Pulseq input
  - Open source: CMRsim, KomaMRI.jl, MR0 / MRTwin, …
  - Closed source: coreMRI, BlochSolver, …

- PyPulseq connects trivially to MR0[1] (EPG simulator)
- Matlab sequences can be simulated directly via MR0 or KomaMRI[2]
  - See “Pulseq News” on Thursday



MR0 Simulation in 5 seconds

1: https://mrsources.github.io/MRzero-Core/
2: https://juliahealth.org/KomaMRI.jl

Further Checks Before Scanning

- Matlab / PyPulseq checks

  seq.checkTiming()

  seq.testReport()

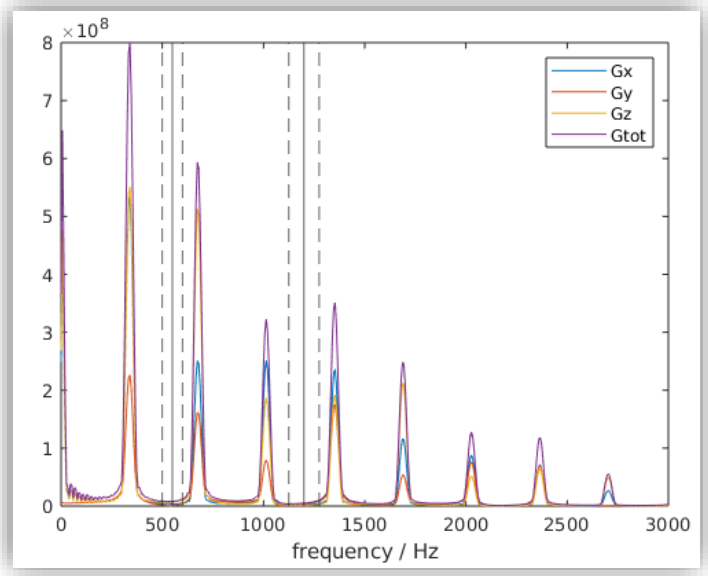
  seq.gradSpectrum()

  seq.calcPNS()

  seq.calcRfPower()

```
Number of blocks: 278
Number of events:
   RF:      138
   Gx:      276
   Gy:      256
   Gz:      278
   ADC:     128
Sequence duration: 0.409910s
TE: 0.001537s
TR: 0.002960s
Flip angle: 5.00°
Unique k-space positions (a.k.a. columns, rows, etc): 256
Unique k-space positions (a.k.a. columns, rows, etc): 128
Dimensions: 2
   Spatial resolution: 1.75 mm
   Spatial resolution: 1.75 mm
Repetitions/slices/contrasts: 1  range: [1 1]
   32768 k-space position(s) repeated 1 times
Cartesian encoding trajectory detected
Block timing check passed successfully
Max. Gradient: 1171875 Hz/m == 27.52 mT/m
Max. Gradient: 1190476 Hz/m == 27.96 mT/m
Max. Gradient: 1153846 Hz/m == 27.10 mT/m
Max. Slew Rate: 5.09511e+09 Hz/m/s == 119.67 T/m/s
Max. Slew Rate: 4.96032e+09 Hz/m/s == 116.51 T/m/s
Max. Slew Rate: 4.90702e+09 Hz/m/s == 115.25 T/m/s
Max. Absolute Gradient: 1937442 Hz/m == 45.51 mT/m
Max. Absolute Slew Rate: 8.63966e+09 Hz/m/s == 202.92 T/m/s
```

- Simulate in vendor environments (GE, Siemens)



gradient spectrogram

Conclusions

- Rapid sequence development
  - Visualization and debugging tools
  - Access to arbitrary shape RF and gradient pulses
- Extended pre-flight checks
  - Hardware safety and performance
  - Subject safety
- Reproducible results
  - Fixed sequence
  - Embedded hash / sequence signature

illustration by chatGPT

Berkin Bilgic
Feng Jia
Jon-Fredrik Nielsen
Marten Veldmann
Niklas Wehkamp
Qingping Chen
Tony Stoecker
Borjan Gagoski
Imam Shaik
Juergen Hennig
Moritz Zaiss
Philipp Amrein
Scott Peltier
Will Grissom
Douglas Noll
Jeff Fessler
Lukas Winter
Naveen Murthy
Qiang Liu
Sebastian Littin
Yogesh Rathi

THANK YOU FOR YOUR ATTENTION!

Pulseq MATLAB

pypulseq Python

harmonized MRI

NIH National Institutes of Health

NIH U24 S120056
NIH 1R01EB032378

EU MRITwins 101078393

Co-funded by the European Union

METROLOGY PARTNERSHIP EURAMET

EURAMET 22HLT02 A4IM